

■ App Feiras Medievais — Portugal

Plano de Desenvolvimento · Junho 2026

Este documento resume o plano estruturado de desenvolvimento de uma aplicação para tracking de Feiras Medievais em Portugal, com base na análise de fontes existentes como *feirasmedievais.pt* e *aondevamos.pt*. O projeto está organizado em cinco fases sequenciais, desde a recolha de dados até ao deploy e manutenção em produção.

Visão Geral da Arquitetura

Camada	Tecnologia / Ferramenta	Responsabilidade
Fontes de dados	feirasmedievais.pt · aondevamos.pt	Origem dos dados de feiras
Scraping / AI	Python · BeautifulSoup · Playwright	Extração automática de dados
Base de dados	Supabase (PostgreSQL)	Armazenamento estruturado
API	Supabase REST (automática)	Acesso aos dados pela app
Frontend / App	Next.js (Web) ou React Native	Interface do utilizador
Deploy	Vercel · GitHub Actions	Hospedagem e automação

Fase 1 — Recolha de Dados (Scraping)

Objetivo: Extrair dados estruturados dos sites fonte e convertê-los em registos georeferenciados prontos para a base de dados.

Passo 1.1 — Análise dos Sites Fonte

Antes de escrever qualquer código de scraping, é essencial perceber como os dados estão organizados nos sites fonte. Esta análise é a base de toda a pipeline de dados.

- Inspeccionar a estrutura HTML de *feirasmedievais.pt* e *aondevamos.pt* com as DevTools do browser.
- Identificar os campos disponíveis em cada site: nome do evento, datas, localização, descrição, imagem, link de detalhe.
- Verificar se existe API pública ou feed RSS — se existir, é preferível ao scraping direto (mais estável e limpo).
- Testar a renderização: perceber se o conteúdo é estático (HTML puro) ou dinâmico (JavaScript/React), pois isso condiciona a ferramenta de scraping a usar.
- Documentar as diferenças de estrutura entre os dois sites (campos distintos, formatos de data, etc.).

Passo 1.2 — Construção do Scraper

O scraper é o coração do projeto na fase inicial. Deve ser robusto, configurável e capaz de lidar com variações nos dados (campos em falta, formatos diferentes, etc.).

- Ferramenta principal:** Python com *BeautifulSoup* para sites estáticos. Se o site renderizar com JavaScript, usar *Playwright* (controla um browser real).
- Campos a extrair por feira:** nome do evento · data início e fim · localidade · distrito · morada · descrição · URL da fonte · URL de imagem.

- Implementar tratamento de erros (site em baixo, estrutura alterada, campos em falta) para que o scraper não quebre silenciosamente.
- Criar um módulo separado por site fonte (ex: `scraper_feirasmedievais.py`, `scraper_aondevamos.py`) para facilitar manutenção.
- Normalizar os dados para um schema comum antes de os enviar para a base de dados (unificar formatos de data, capitalização, etc.).
- Implementar logging detalhado para rastrear o que foi encontrado, inserido ou ignorado em cada execução.

Passo 1.3 — Geocoding

Para mostrar as feiras no mapa, é necessário converter moradas ou nomes de localidades em coordenadas GPS (latitude/longitude). Este processo chama-se geocoding.

- Usar a API **Nominatim** do OpenStreetMap (gratuita, sem necessidade de conta) como primeira opção.
- Em alternativa, usar a **Google Maps Geocoding API** (paga, mais precisa) se a qualidade do Nominatim for insuficiente.
- Guardar as coordenadas na base de dados junto ao registo da feira para evitar geocoding repetido.
- Implementar cache local para localidades já geocodificadas (uma localidade como 'Óbidos' só precisa ser geocodificada uma vez).
- Tratar casos sem resultado (localidades muito pequenas ou com nomes ambíguos) com fallback manual ou marcação para revisão.

Fase 2 — Base de Dados

Objetivo: Criar uma base de dados robusta e bem estruturada que sirva como única fonte de verdade da aplicação.

Passo 2.1 — Escolha da Stack

A escolha da base de dados determina o esforço de desenvolvimento do backend. Para este projeto, a recomendação é **Supabase**:

Porquê Supabase: é PostgreSQL gerido na cloud, com API REST automática (sem precisar de escrever backend), autenticação integrada, painel de administração web, e free tier generoso (suficiente para começar). Alternativa NoSQL seria Firebase, mas o modelo relacional do Supabase é mais adequado para dados estruturados como feiras.

Passo 2.2 — Schema da Tabela Principal

A tabela central do sistema deve ter os seguintes campos:

Campo	Tipo	Descrição
id	UUID / serial	Identificador único da feira
nome	TEXT	Nome do evento (ex: Feira Medieval de Óbidos)
descricao	TEXT	Descrição do evento
data_inicio	DATE	Data de início da feira
data_fim	DATE	Data de fim da feira
localidade	TEXT	Cidade/vila onde decorre

distrito	TEXT	Distrito (para filtros regionais)
lat	FLOAT	Latitude (do geocoding)
lng	FLOAT	Longitude (do geocoding)
imagem_url	TEXT	URL da imagem/cartaz do evento
fonte_url	TEXT	URL original do site fonte
criado_em	TIMESTAMP	Data de inserção na BD
atualizado_em	TIMESTAMP	Data da última atualização

Passo 2.3 — Automatização da Atualização

O scraper não deve ser executado manualmente. Deve correr de forma autónoma e periódica para manter os dados atualizados.

- Configurar um **GitHub Actions workflow** com agendamento semanal (*schedule: cron*) que executa o scraper automaticamente.
- Implementar lógica de **upsert**: comparar os dados novos com os existentes na BD e apenas inserir ou atualizar o que mudou (evita duplicados).
- Enviar notificação (email ou webhook) em caso de falha do scraper para deteção precoce de problemas.
- Guardar um log de cada execução (quantas feiras encontradas, inseridas, atualizadas) para monitorização.

Fase 3 — Backend / API

Objetivo: Expor os dados de forma estruturada para que o frontend os possa consumir com filtros e pesquisa.

Passo 3.1 — API de Dados

Se for utilizado o Supabase, a API REST é gerada automaticamente e esta fase é significativamente simplificada. Mesmo assim, existem tarefas importantes de configuração:

- **Supabase API automática:** ativar e configurar as políticas de Row Level Security (RLS) — definir que dados são públicos (leitura) e quais requerem autenticação (escrita).
- **Endpoints essenciais que devem funcionar:**
 - → *GET /feiras* — lista todas as feiras, ordenadas por data de início (mais próximas primeiro).
 - → *GET /feiras?distrito=eq.Lisboa* — filtro por distrito.
 - → *GET /feiras?data_inicio=gte.2026-06-01* — feiras a partir de uma data.
 - → *GET /feiras?lat=38.7&lng=-9.1&raio=50* — feiras num raio de X km (requer função PostGIS no Supabase).
- Configurar **CORS** para permitir pedidos do domínio da app.
- Criar **índices** na BD nos campos mais filtrados (distrito, data_inicio, lat/lng) para melhor performance.
- Documentar os endpoints disponíveis para referência durante o desenvolvimento do frontend.

Fase 4 — Frontend / Aplicação

Objetivo: Criar a interface que os utilizadores vão usar para descobrir e explorar feiras medievais.

Passo 4.1 — Escolha do Tipo de App

Opção	Prós	Contras	Recomendação
Web App (Next.js)	Rápido de desenvolver Sem necessidade de app store SEO nativo	Menos 'nativa' Limitações offline	✓ Para MVP
React Native (Expo)	iOS + Android com um código Experiência nativa	Setup mais complexo Testes em dispositivo	Fase 2+
PWA (Web Progressiva)	Instalável no telemóvel Funciona no browser	Notificações limitadas Suporte variável por browser	Alternativa futura

Passo 4.2 — Funcionalidades MVP (Produto Mínimo Viável)

O MVP deve ser simples mas funcional: permite validar a ideia sem gastar demasiado tempo. As funcionalidades abaixo são o mínimo necessário para uma app útil:

- **Lista de feiras por data** — mostrar as feiras ordenadas cronologicamente, com as próximas primeiro. Cada item deve mostrar: nome, data, localidade e imagem.
- **Filtro por distrito/região** — dropdown ou botões para filtrar por região de Portugal (Norte, Centro, Lisboa, Alentejo, Algarve, Ilhas).
- **Mapa interativo com pins das feiras** — usar *Leaflet.js* com tiles do OpenStreetMap (gratuito). Cada pin abre um popup com info básica da feira.
- **Página de detalhe de cada feira** — ao clicar numa feira, abrir uma página com todos os detalhes: descrição completa, datas, localização, imagem, link para o site original.
- **Design responsivo** — a app deve funcionar bem tanto em desktop como em telemóvel (mobile-first).
- **Página de 'sobre'** — explicar o projeto, as fontes de dados e como submeter correções.

Passo 4.3 — Funcionalidades Futuras (Pós-MVP)

Estas funcionalidades não fazem parte do MVP mas devem ser consideradas na arquitetura desde o início para facilitar a sua implementação futura:

- **Notificações de proximidade** — alertar o utilizador quando uma feira está perto da sua localização atual.
- **Favoritos / 'Quero ir'** — sistema de marcadores para o utilizador guardar feiras do seu interesse (requer autenticação).
- **Submissão de feiras pela comunidade** — formulário para utilizadores sugerirem feiras não listadas (com moderação).
- **Integração com Google Calendar** — botão para adicionar uma feira diretamente ao calendário do utilizador.
- **App móvel nativa** — migrar para React Native/Expo após validação do conceito na versão web.
- **Histórico de feiras** — arquivo de edições passadas para referência e análise de tendências.

Fase 5 — Deploy e Manutenção

Objetivo: Colocar a aplicação em produção de forma estável, segura e com custos mínimos.

- **Frontend — Vercel:** plataforma de deploy automático para Next.js. Basta ligar ao repositório GitHub e cada push para a branch principal faz deploy automático. Free tier suficiente para começar.

- **Base de dados — Supabase:** free tier inclui 500MB de armazenamento e 2GB de largura de banda por mês. Suficiente para o volume de dados de feiras medievais.
- **Scraper — GitHub Actions:** configurar um workflow com schedule semanal (ex: domingo de madrugada). Gratuito para repositórios públicos e com limite generoso para privados.
- **Domínio:** registar um domínio como *feirasmedievais.app* ou *feiraspt.com*. Custo anual de ~10-15€. Ligar ao Vercel via DNS.
- **Monitorização:** ativar alertas de uptime (ex: UptimeRobot — gratuito) e monitorizar logs do Supabase para detetar erros de acesso ou performance.
- **Backup:** configurar backup automático da base de dados Supabase (disponível no free tier) para evitar perda de dados.

Roadmap Resumido

Período	Fase	Entregável
Semana 1–2	Fase 1 + Fase 2	Scraper funcional + BD populada com feiras reais
Semana 3–4	Fase 3 + Fase 4 (MVP)	App web com lista, filtros e mapa online
Semana 5–6	Fase 4 (polish) + Fase 5	App polida, deploy público, domínio configurado
Após lançamento	Funcionalidades futuras	Notificações, favoritos, submissão comunitária

Nota: Os dados são obtidos via scraping de sites públicos e podem ter algum desfasamento temporal. A atualização automática semanal minimiza este risco. Fontes: feirasmedievais.pt · aondevamos.pt

Gerado por Claude · Junho 2026